

4. Предобработка данных

По условию задания необходимо:

- Преобразовать форматы данных, если это требуется для анализа.
- Принять решение о способе обработки пропусков. Обосновать его и выполнить заполнение или удаление.
- Выявить выбросы. Обосновать решение об их корректировке или удалении и выполнить обработку.
- Применить разные виды масштабирования и трансформации как минимум к двум числовым признакам. Прокомментировать результаты.
- Преобразовать категориальный признак. Обосновать выбранный способ и объяснить результат.

Каждый шаг должен сопровождаться содержательным комментарием. Ниже приведены план обработки этого датасета, расчёты и выводы.

4.1. План обработки и рабочая копия

Продолжим анализ из пунктов 1–3. Целевая переменная: `salary_usd`.

План работы:

1. Преобразовать даты и привести варианты названий к единым категориям.
2. Обработать ошибки и подозрительные значения. Записать изменения в журнал.
3. Разделить данные на обучающую и отложенную части. Затем заполнить пропуски в признаках.
4. Сравнить три способа масштабирования и два нелинейных преобразования для двух числовых признаков.
5. Закодировать категории. В пункте 5 создать шесть признаков, проверить их связь с зарплатой и изучить PCA.

Используем методы из лекций: лекция 1, слайды 45–59 (разбиение и утечки); лекция 2, слайды 47–53 (корреляции); лекция 3, слайды 7–18, 26–45 (очистка, масштабирование и кодирование); лекция 4, слайды 8–30 (PCA). PDF доступны в [папке курса](#).

Для запуска положите `ai_job_dataset_upd.csv` рядом с ноутбуком. Зависимости указаны в `requirements.txt`. Выполняйте ячейки по порядку, начиная с первого пункта.

Цель. Подключить библиотеки и задать общие настройки расчётов. Ячейка показывает версии библиотек и определяет вспомогательные функции для поправки Холма, оформления таблиц и графиков.

```
from pathlib import Path
import hashlib
import platform
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import scipy
from scipy import stats
import sklearn
from sklearn.model_selection import GroupShuffleSplit
from sklearn.preprocessing import (
    MinMaxScaler,
    StandardScaler,
    RobustScaler,
    OrdinalEncoder,
    OneHotEncoder,
)
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.decomposition import PCA
from IPython.display import display
```

```

RANDOM_STATE = 42
ALPHA = 0.05
LEVELS = ["EN", "MI", "SE", "EX"]
SIZES = ["S", "M", "L"]
LEVEL_MAP = dict(zip(LEVELS, range(4)))
SIZE_MAP = dict(zip(SIZES, range(3)))
BLUE, GOLD, GRAY = "#31688e", "#b98524", "#68727a"
sns.set_theme(style="whitegrid", font="DejaVu Sans", font_scale=1.02)
plt.rcParams.update({"figure.dpi": 110, "axes.spines.top": False, "axes.spines.right": False})
pd.set_option("display.max_columns", 12)
pd.set_option("display.max_rows", 25)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

def holm(pvalues):
    """Поправка Холма для нескольких одновременных проверок."""
    pvalues = np.asarray(pvalues, dtype=float)
    order = np.argsort(pvalues)
    adjusted = np.empty(len(pvalues))
    previous = 0
    for rank, position in enumerate(order):
        corrected = (len(pvalues) - rank) * pvalues[position]
        previous = max(previous, corrected)
        adjusted[position] = min(1, previous)
    return adjusted

def p_text(p):
    return "<1e-300" if p < 1e-300 else f"{p:.3g}"

def show_stats(table, pcols=()):
    formats = {c: p_text for c in pcols}
    display(table.style.format(formats, precision=3, thousands=" "))

def finish():
    plt.tight_layout()
    plt.show()

display(
    pd.Series(
        {
            "Python": platform.python_version(),
            "pandas": pd.__version__,
            "NumPy": np.__version__,
            "SciPy": scipy.__version__,
            "scikit-learn": sklearn.__version__,
            "seaborn": sns.__version__,
        },
        name="Версия",
    )
)

```

```

Python          3.11.5
pandas          2.2.1
NumPy           1.26.4
SciPy           1.11.4
scikit-learn    1.4.1.post1
seaborn         0.13.2
Name: Версия, dtype: object

```

Цель. Проверить исходный CSV и создать рабочую копию. Таблицы сравниваются целиком, подсчитываются дубликаты. Крайние зарплаты показываются отдельно для поиска аномалий. Контрольная сумма сохраняется для проверки файла в конце.

```

raw = df.copy(deep=True)
DATA_PATH = Path("ai_job_dataset_upd.csv")
source_sha256 = hashlib.sha256(DATA_PATH.read_bytes()).hexdigest()
source_matches = raw.equals(pd.read_csv(DATA_PATH, sep=";"))
print("Рабочая копия совпадает с CSV:", source_matches)
print("Размер таблицы:", raw.shape)

```

```
print("Полные дубликаты:", raw.duplicated().sum())
print("Повторы job_id:", raw.job_id.duplicated().sum())

# Крайние значения показывают разрывы в годовых зарплатах.
salary_edges = pd.DataFrame(
    {
        "Нижний край, USD/год": raw.salary_usd.nsmallest(14).to_numpy(),
        "Верхний край, USD/год": raw.salary_usd.nlargest(14).to_numpy(),
    }
)
display(salary_edges)
```

Рабочая копия совпадает с CSV: True
 Размер таблицы: (15000, 19)
 Полные дубликаты: 0
 Повторы job_id: 0

	Нижний край, USD/год	Верхний край, USD/год
0	-999.000	99,999,999.000
1	-78.000	10,000,000.000
2	-1.000	9,383,302.000
3	0.000	399,095.000
4	0.000	398,084.000
5	1.000	394,917.000
6	3.000	390,292.000
7	5.000	388,754.000
8	10.000	383,142.000
9	12.000	381,575.000
10	45.000	379,418.000
11	32,519.000	372,206.000
12	32,542.000	371,087.000
13	32,587.000	370,038.000

Правила очистки. Дубликатов нет. Удаляем пробелы по краям строк, заменяем US и UK полными названиями стран. Приводим отрасли к единым категориям, считаем Ph сокращением от PhD. Опираемся на имеющийся словарь. Значение кодов 8888, 9999 и 6 неизвестно, поэтому ставим пропуски.

salary_usd содержит **годовую зарплату в долларах США**. Нулевые и отрицательные значения не используем. После 1, 3, 5, 10, 12 и 45 USD следующая зарплата равна 32 519 USD. В верхнем хвосте после 399 095 USD идут 9 383 302, 10 000 000 и 99 999 999 USD. Разрывы видны в таблице выше.

Для этого файла подозрительными считаем зарплаты **ниже 10 000 или не ниже 500 000 USD в год**. Это рабочие границы поиска аномалий, а не допустимые зарплаты на рынке. Они отделяют единицы долларов и три многомиллионных значения от основной группы. Для анализа оставляем **$10\,000 \leq \text{salary_usd} < 500\,000$** и ниже проверяем другие границы. Само прохождение фильтра ещё не подтверждает достоверность записи.

Правильные значения неизвестны. Не домножаем зарплаты на 12 или 1 000 и не подставляем среднее. Ставим NaN в рабочей копии. Исходные значения остаются в CSV и журнале изменений: при появлении первоисточника их можно проверить. Выход за усы ящичной диаграммы сам по себе не повод удалять запись.

Балл соцпакета по словарю должен быть от 1 до 10. Для длины описания принимаем рабочий диапазон от 10 до 1 000 000 символов. Отрицательная длина невозможна, а 0, 1 или 6 символов выглядят сомнительно. Многомиллионные значения тоже сильно отличаются от остальных. Восстановить их нельзя, поэтому ставим NaN. Исходный df сохраняем, изменения записываем в журнал.

Цель. Применить правила очистки: исправить категории, преобразовать даты, заменить сомнительные числа пропусками. Изменения записываются в журнал. Проверки категорий, стажа и дат сохраняются в `data_checks` и выводятся в конце работы.

```
SALARY_MIN = 10_000 # USD в год; рабочая граница поиска аномалий.
SALARY_CAP = 500_000 # Допущение для основного анализа, не универсальный предел оплаты.
checked = raw.copy(deep=True)
changes = []

def record_replace(column, new_values, reason):
    old = checked[column].copy()
    changed = ~((old == new_values) | (old.isna() & new_values.isna()))
    for idx in checked.index[changed.fillna(True)]:
        changes.append(
            {
                "job_id": checked.at[idx, "job_id"],
                "column": column,
                "before": str(old.at[idx]),
                "after": str(new_values.at[idx]),
                "reason": reason,
            }
        )
    checked[column] = new_values

for col in checked.select_dtypes("object"):
    record_replace(col, checked[col].str.strip(), "Пробелы по краям строки")
aliases = {
    "employee_residence": {"US": "United States", "UK": "United Kingdom", "8888": np.nan},
    "education_required": {"Associate.": "Associate", "Ph": "PhD", "6": np.nan},
    "industry": {
        "Retail_": "Retail",
        "manufacturing": "Manufacturing",
        "Transport": "Transportation",
        "Governmnt": "Government",
        "Telecom": "Telecommunications",
        "9999": np.nan,
    },
}
for col, mapping in aliases.items():
    record_replace(col, checked[col].replace(mapping), "Явный словарь исправлений")

for col in ["posting_date", "application_deadline"]:
    checked[col] = pd.to_datetime(checked[col], format="%Y-%m-%d", errors="coerce")

numeric_invalid = {
    "salary_usd": (checked.salary_usd < SALARY_MIN) | (checked.salary_usd >= SALARY_CAP),
    "benefits_score": checked.benefits_score.notna() & ~checked.benefits_score.between(1, 10),
    "job_description_length": checked.job_description_length.notna()
    & ~checked.job_description_length.between(10, 1_000_000),
}
for col, mask in numeric_invalid.items():
    record_replace(
        col, checked[col].mask(mask), "Недопустимо или подозрительно по объявленному правилу"
    )

audit = pd.DataFrame(changes)
display(audit.groupby(["column", "reason"]).size().rename("Изменений").to_frame())
display(audit[audit.column.isin(numeric_invalid)][["job_id", "column", "before", "after"]])
salary_valid = checked.salary_usd.dropna()
education_levels = ["Associate", "Bachelor", "Master", "PhD"]
data_checks = {
    "Нет повторов job_id": checked.job_id.is_unique,
    "Грейды есть в словаре": checked.experience_level.isin(LEVELS).all(),
    "Размеры компаний есть в словаре": checked.company_size.isin(SIZES).all(),
    "Формат работы имеет известный код": checked.remote_ratio.isin([0, 50, 100]).all(),
    "Уровни образования есть в словаре": checked.education_required.dropna()
    .isin(education_levels)
    .all(),
    "Стаж находится между 0 и 60 годами": checked.years_experience.between(0, 60).all(),
    "Дедлайн не раньше публикации": (
        checked.application_deadline >= checked.posting_date
    )
}
```

```
    ).all(),  
}
```

		Изменений
column	reason	
benefits_score	Недопустимо или подозрительно по объявленному правилу	1
education_required	Пробелы по краям строки	1
	Явный словарь исправлений	3
employee_residence	Пробелы по краям строки	1
	Явный словарь исправлений	3
industry	Пробелы по краям строки	2
	Явный словарь исправлений	6
job_description_length	Недопустимо или подозрительно по объявленному правилу	6
salary_usd	Недопустимо или подозрительно по объявленному правилу	14

	job_id	column	before	after
16	AI00123	salary_usd	10.0	nan
17	AI00192	salary_usd	-78.0	nan
18	AI00258	salary_usd	99999999.0	nan
19	AI00638	salary_usd	3.0	nan
20	AI00703	salary_usd	5.0	nan
21	AI00795	salary_usd	-999.0	nan
22	AI00947	salary_usd	10000000.0	nan
23	AI01451	salary_usd	45.0	nan
24	AI01856	salary_usd	12.0	nan
25	AI02040	salary_usd	-1.0	nan
26	AI02223	salary_usd	9383302.0	nan
27	AI02421	salary_usd	1.0	nan
28	AI02528	salary_usd	0.0	nan
29	AI02714	salary_usd	0.0	nan
30	AI10988	benefits_score	9999.0	nan
31	AI09636	job_description_length	1.0	nan
32	AI09643	job_description_length	0.0	nan
33	AI09782	job_description_length	6.0	nan
34	AI09848	job_description_length	9999999.0	nan
35	AI13048	job_description_length	99999999.0	nan
36	AI13051	job_description_length	-3.0	nan

Цель. Проверить последствия очистки. Ячейка сравнивает число пропусков до и после обработки, показывает исправленные категории и диапазоны дат. Отдельно подсчитываются вакансии с дедлайном раньше даты публикации.

```
display(  
    pd.DataFrame(  
        {"До обработки": raw.isna().sum(), "После проверки": checked.isna().sum()})
```

```

).loc[lambda t: t.sum(axis=1) > 0]
)
display(audit.loc[audit.column.isin(["employee_residence", "education_required", "industry"])])
date_summary = pd.DataFrame(
    {
        c: [checked[c].min(), checked[c].max(), checked[c].isna().sum()]
        for c in ["posting_date", "application_deadline"]
    },
    index=["Минимум", "Максимум", "Пропуски"],
)
display(date_summary)
print(
    "Дедлайн раньше публикации:", (checked.application_deadline < checked.posting_date).sum()
)

```

	До обработки	После проверки
salary_usd	10	24
employee_residence	2	3
education_required	5	6
industry	3	4
job_description_length	8	14
benefits_score	16	17

	job_id	column	before	after	reason
0	AI10023	employee_residence	United States	United States	Пробелы по краям строки
1	AI05962	education_required	Associate.	Associate.	Пробелы по краям строки
2	AI00192	industry	Government	Government	Пробелы по краям строки
3	AI05867	industry	Retail	Retail	Пробелы по краям строки
4	AI10035	employee_residence	US	United States	Явный словарь исправлений
5	AI10120	employee_residence	UK	United Kingdom	Явный словарь исправлений
6	AI10262	employee_residence	8888	nan	Явный словарь исправлений
7	AI00097	education_required	6	nan	Явный словарь исправлений
8	AI05962	education_required	Associate.	Associate	Явный словарь исправлений
9	AI06006	education_required	Ph	PhD	Явный словарь исправлений
10	AI00253	industry	Retail_	Retail	Явный словарь исправлений
11	AI00442	industry	manufacturing	Manufacturing	Явный словарь исправлений
12	AI00519	industry	Transport	Transportation	Явный словарь исправлений
13	AI00927	industry	Governmnt	Government	Явный словарь исправлений
14	AI06095	industry	Telecom	Telecommunications	Явный словарь исправлений
15	AI06513	industry	9999	nan	Явный словарь исправлений

	posting_date	application_deadline
Минимум	2024-01-01 00:00:00	2024-01-16 00:00:00
Максимум	2025-04-30 00:00:00	2025-07-11 00:00:00
Пропуски	0	0

Дедлайн раньше публикации: 0

После обработки для анализа сохранена зарплата 14 976 вакансий. Неизвестная или сомнительная зарплата: 24 строки. Пропусков в балле соцпакета: 17. Пропусков в длине описания: 14.

Даты преобразованы успешно. Срок подачи заявок нигде не заканчивается раньше публикации. В файле есть вакансии за 2024 год и январь–апрель 2025 года. Используем весь выданный файл. По нему нельзя оценить весь рынок за 2025 год. После исправлений US и United States относятся к одной стране. Аналогично объединены другие варианты категорий.

4.2. Проверка порога зарплат

Сравним несколько нижних и верхних порогов. Отдельно сохраним все положительные зарплатy. Посмотрим, как меняются статистики. Качество прогноза здесь не оценивается.

Цель. Проверить чувствительность выводов к границам зарплатy. Для четырёх вариантов отбора сравниваются число строк, среднее, медиана и корреляция Спирмена со стажем.

```
variants = [
    ("Все положительные", 0, np.inf),
    ("Основной: 10 000–500 000", 10_000, 500_000),
    ("Шире: 5 000–1 000 000", 5_000, 1_000_000),
    ("Выше нижняя граница: 30 000–500 000", 30_000, 500_000),
]
sensitivity_rows = []
for name, lower, upper in variants:
    selected_rows = (raw.salary_usd > 0) & (raw.salary_usd >= lower) & (raw.salary_usd < upper)
    part = raw.loc[selected_rows]
    correlation = stats.spearmanr(part.salary_usd, part.years_experience).statistic
    sensitivity_rows.append(
        {
            "Вариант, USD/год": name,
            "Строки": len(part),
            "Среднее, USD/год": part.salary_usd.mean(),
            "Медиана, USD/год": part.salary_usd.median(),
            "ρ зарплатy и стажа": correlation,
        }
    )
sensitivity = pd.DataFrame(sensitivity_rows).set_index("Вариант, USD/год")
display(sensitivity)
```

	Строки	Среднее, USD/год	Медиана, USD/год	ρ зарплатy и стажа
Вариант, USD/год				
Все положительные	14985	123,248.049	99,642.000	0.791
Основной: 10 000–500 000	14976	115,350.470	99,658.500	0.792
Шире: 5 000–1 000 000	14976	115,350.470	99,658.500	0.792
Выше нижняя граница: 30 000–500 000	14976	115,350.470	99,658.500	0.792

При сохранении всех положительных зарплат среднее равно 123 248 USD. При основном варианте отбора оно равно 115 350 USD. Медиана после обработки составляет 99 658,5 USD. Связь зарплатy со стажем почти не меняется: $\rho = 0,792$.

Варианты 5 000–1 000 000, 10 000–500 000 и 30 000–500 000 USD в год оставляют одинаковые 14 976 строк. Пороги попадают в разрывы между группами зарплат, поэтому здесь выбор конкретной границы не влияет на результат. Три экстремальные зарплатy заметно меняют среднее. Медиана и ранговая корреляция устойчивее. Исключение этих строк остаётся нашим допущением. Если появится доступ к первоисточнику, значения нужно проверить.

4.3. Пропуски и разделение данных

В `benefits_score` и `job_description_length` заполняем пропуски медианами обучающей части. Добавляем индикаторы, которые показывают, где значение отсутствовало. Для пропусков в категориях используем `Unknown`. Так мы не приписываем записи выдуманную категорию.

Пропуски в salary_usd не заполняем. Иначе при обучении модель получила бы придуманный правильный ответ. Строки без используемой в анализе зарплаты сохраняем отдельно. Позже для них можно будет получить прогноз. При расчёте корреляций используем только пары с известными значениями.

До обучения преобразований делим данные по названиям компаний. В отложенную часть попадают 20% из 16 названий с округлением вверх: четыре компании. Это около 25% строк. Названия компаний между частями не пересекаются. Названия могут быть условными, поэтому независимость работодателей не гарантирована. Разбиение позволяет проверить применение преобразований к новым данным. Для независимой оценки качества модели оно не подходит: предыдущий анализ уже использовал весь файл.

На слайде 45 лекции 3 есть правило: «обучаем (fit) все шаги предобработки только на train». Поэтому мы считаем медианы и параметры масштабирования на обучающей части, а к отложенной применяем уже готовые преобразования. Здесь держим себя в руках: в test за подсказками не заглядываем.

Цель. Разделить данные по компаниям с помощью GroupShuffleSplit. Строки без используемой зарплаты сохранить отдельно. Проверить учёт всех строк и отсутствие общих компаний и вакансий между частями.

```
usable = checked.loc[checked.salary_usd.notna()].copy()
unlabeled = checked.loc[checked.salary_usd.isna()].copy()
splitter = GroupShuffleSplit(n_splits=1, test_size=0.2, random_state=RANDOM_STATE)
train_pos, test_pos = next(splitter.split(usable, groups=usable.company_name))
train = usable.iloc[train_pos].copy()
test = usable.iloc[test_pos].copy()
display(
    pd.DataFrame(
        {
            "Строки": [len(raw), len(usable), len(unlabeled), len(train), len(test)],
            "Описание": [
                "Исходник",
                "Наблюдаемая цель",
                "Неизвестная цель",
                "Обучающая часть",
                "Отложенная часть",
            ],
        }
    )
)
```

	Строки	Описание
0	15000	Исходник
1	14976	Наблюдаемая цель
2	24	Неизвестная цель
3	11195	Обучающая часть
4	3781	Отложенная часть

Обучающая часть: 11 195 строк, число компаний: 12. Отложенная часть: 3 781 строка, число компаний: 4. Без используемой в анализе зарплаты: 24 строки. Они сохранены отдельно. Пересечений job_id и названий компаний нет.

Медиана устойчива к крайним значениям, но заполнение пропусков уменьшает разброс. Истинные значения при этом остаются неизвестными. Поэтому заполненные данные используем для подготовки матрицы, а связи изучаем по исходно известным парам. Даже редкие пропуски могут возникать закономерно. Их индикаторы сохраняют эту информацию для модели.

4.4. Масштабирование двух числовых признаков

Для years_experience и job_description_length сравним MinMaxScaler, StandardScaler и RobustScaler. В каждом Pipeline сначала заполняем пропуски медианой, затем меняем масштаб. Параметры определяем только по

обучающей части. Все три преобразования линейны. Они меняют шкалу, но **не делают распределение нормальным**. Зарплату в матрицу признаков не включаем.

Цель. Сравнить три способа масштабирования двух признаков. Параметры оцениваются на train и применяются к test. Таблица показывает средние, диапазоны и разброс. Гистограммы не повторяем: их форма сохраняется.

```
scale_columns = ["years_experience", "job_description_length"]
scalers = {"Min-Max": MinMaxScaler(), "Standard": StandardScaler(), "Robust": RobustScaler()}
scaled_train, scaled_test = {}, {}
scale_pipelines = {}
scale_rows = []
for name, scaler in scalers.items():
    pipe = Pipeline([("imputer", SimpleImputer(strategy="median")), ("scaler", scaler)])
    a = pipe.fit_transform(train[scale_columns])
    b = pipe.transform(test[scale_columns])
    scaled_train[name] = pd.DataFrame(a, index=train.index, columns=scale_columns)
    scaled_test[name] = pd.DataFrame(b, index=test.index, columns=scale_columns)
    scale_pipelines[name] = pipe
    for col in scale_columns:
        for split, data in [("train", scaled_train[name]), ("test", scaled_test[name])]:
            x = data[col]
            scale_rows.append(
                {
                    "Метод": name,
                    "Часть": split,
                    "Признак": col,
                    "Среднее": x.mean(),
                    "SD (ddof=0)": x.std(ddof=0),
                    "Медиана": x.median(),
                    "IQR": x.quantile(0.75) - x.quantile(0.25),
                    "min": x.min(),
                    "max": x.max(),
                }
            )
scale_summary = pd.DataFrame(scale_rows)
display(scale_summary)
```

	Метод	Часть	Признак	Среднее	SD (ddof=0)	Медиана	IQR	min	max
0	Min-Max	train	years_experience	0.333	0.292	0.263	0.421	0.000	1.000
1	Min-Max	test	years_experience	0.317	0.290	0.211	0.421	0.000	1.000
2	Min-Max	train	job_description_length	0.503	0.288	0.506	0.498	0.000	1.000
3	Min-Max	test	job_description_length	0.499	0.289	0.506	0.498	0.000	1.000
4	Standard	train	years_experience	0.000	1.000	-0.239	1.440	-1.139	2.280
5	Standard	test	years_experience	-0.055	0.991	-0.419	1.440	-1.139	2.280
6	Standard	train	job_description_length	-0.000	1.000	0.012	1.730	-1.746	1.727
7	Standard	test	job_description_length	-0.012	1.002	0.010	1.729	-1.746	1.727
8	Robust	train	years_experience	0.166	0.695	0.000	1.000	-0.625	1.750
9	Robust	test	years_experience	0.128	0.688	-0.125	1.000	-0.625	1.750
10	Robust	train	job_description_length	-0.007	0.578	0.000	1.000	-1.016	0.991
11	Robust	test	job_description_length	-0.014	0.579	-0.001	0.999	-1.016	0.991

На обучающей части MinMaxScaler даёт диапазон от 0 до 1. После StandardScaler среднее равно 0, стандартное отклонение равно 1. После RobustScaler медиана равна 0, межквартильный размах равен 1.

В отложенной части эти свойства могут не сохраняться. Если новое значение выходит за обучающий диапазон, MinMaxScaler может дать число меньше 0 или больше 1. Линейные преобразования сохраняют форму распределения, поэтому здесь достаточно таблицы.

Для PCA ниже используем StandardScaler. Так длина описания не будет преобладать из-за больших численных значений. RobustScaler полезен, если допустимые крайние значения сохранены. Ошибки в данных всё равно нужно обрабатывать отдельно.

4.5. Нелинейные преобразования: log1p и квадратный корень

Для тех же двух признаков сравним исходную шкалу, $\log_1p(x)$ и $\sqrt{x}$. Все значения неотрицательны. Рассчитаем асимметрию и эксцесс. У этих преобразований нет подбираемых параметров. Медианы для заполнения пропусков уже получены на обучающей части. Уменьшение асимметрии само по себе не гарантирует улучшения прогноза.

Цель. Сравнить асимметрию и эксцесс двух признаков до и после $\log_1p$ и извлечения квадратного корня. Две гистограммы иллюстрируют изменение формы распределения стажа после $\log_1p$. Результаты всех шести вариантов приведены в таблице.

```
imputer = scale_pipelines["Standard"].named_steps["imputer"]
imputed_train = imputer.transform(train[scale_columns])
imputed_test = imputer.transform(test[scale_columns])
transforms = {"Исходный": lambda x: x, "log1p": np.log1p, "sqrt": np.sqrt}
transform_rows = []
for column_number, column in enumerate(scale_columns):
    for name, transform in transforms.items():
        values = transform(imputed_train[:, column_number])
        transform_rows.append(
            {
                "Признак": column,
                "Метод": name,
                "Асимметрия": stats.skew(values, bias=False),
                "Эксцесс": stats.kurtosis(values, bias=False),
            }
        )
transform_summary = pd.DataFrame(transform_rows).set_index(["Признак", "Метод"])
display(transform_summary)

# Для иллюстрации изменения формы достаточно стажа до и после log1p.
experience = imputed_train[:, 0]
fig, axes = plt.subplots(1, 2, figsize=(9, 3.5), sharey=True)
axes[0].hist(experience, bins=25, color=BLUE)
axes[0].set(title="Стаж до преобразования", xlabel="Стаж, лет", ylabel="Число вакансий")
axes[1].hist(np.log1p(experience), bins=25, color=BLUE)
axes[1].set(title="Стаж после log1p", xlabel="log(1 + стаж)")
finish()
log_train = np.log1p(imputed_train)
log_test = np.log1p(imputed_test)
```

		Асимметрия	Эксцесс
Признак	Метод		
years_experience	Исходный	0.761	-0.556
	log1p	-0.364	-0.921
	sqrt	-0.112	-0.863
job_description_length	Исходный	-0.014	-1.197
	log1p	-0.566	-0.752
	sqrt	-0.277	-1.072



years_experience: исходная асимметрия 0.76, log1p -0.36, sqrt -0.11; job_description_length: исходная асимметрия -0.01, log1p -0.57, sqrt -0.28.

Для стажа оба преобразования уменьшают правую асимметрию. Длина описания после очистки распределена примерно равномерно. Логарифм создаёт отрицательную асимметрию. Поэтому автоматически применять его к длине описания не будем. В основной матрице оставляем исходные числовые признаки и стандартизируем их. Для нулевого стажа подходит log1p, поскольку обычный $\log(0)$ не определён.

4.6. Кодирование категорий

Сначала покажем порядковое кодирование: EN < MI < SE < EX и S < M < L. Числовые коды сохраняют порядок. Равные расстояния между категориями из этого не следуют.

В основной матрице используем one-hot encoding (ONE), в том числе для грейда. При таком кодировании линейная модель сможет оценить отдельный эффект каждой категории. Образование тоже кодируем через ONE: категорию Unknown нельзя поставить на определённую ступень образования.

job_id и company_name в признаки не включаем. Название компании нужно только для разбиения данных. Из навыков и дат создадим новые признаки в пункте 5.

Цель. Показать порядковое кодирование и ONE. Обработчик обучается на train и преобразует обе части. Итоговые проверки контролируют состав столбцов, пропуски и исключение зарплаты и идентификаторов.

```
ordinal = OrdinalEncoder(
    categories=[LEVELS, SIZES], handle_unknown="use_encoded_value", unknown_value=-1
)
ordinal_train = ordinal.fit_transform(train[["experience_level", "company_size"]])
ordinal_test = ordinal.transform(test[["experience_level", "company_size"]])
display(pd.DataFrame({"Грейд": LEVELS, "Код": [0, 1, 2, 3]}))
display(pd.DataFrame({"Размер": SIZES, "Код": [0, 1, 2]}))
remote_encoder = OneHotEncoder(sparse_output=False, handle_unknown="ignore", dtype=int)
remote_encoder.fit(train[["remote_ratio"]])
display(
    pd.DataFrame(
        remote_encoder.transform(pd.DataFrame({"remote_ratio": [0, 50, 100]})),
        columns=remote_encoder.get_feature_names_out(),
        index=["Офис", "Гибрид", "Удалённо"],
    )
)

base_numeric = ["years_experience", "benefits_score", "job_description_length"]
base_categorical = [
    "experience_level",
    "company_size",
    "remote_ratio",
    "education_required",
    "industry",
    "employee_residence",
```

```

    "company_location",
    "employment_type",
    "job_title",
    "salary_currency",
]

def make_preprocessor(numeric, categorical):
    numeric_steps = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="median", add_indicator=True)),
            ("scaler", StandardScaler()),
        ]
    )
    category_steps = Pipeline(
        [
            ("imputer", SimpleImputer(strategy="constant", fill_value="Unknown")),
            ("encoder", OneHotEncoder(handle_unknown="ignore", sparse_output=False)),
        ]
    )
    return ColumnTransformer(
        [
            ("numeric", numeric_steps, numeric),
            ("category", category_steps, categorical),
        ],
        remainder="drop",
        verbose_feature_names_out=True,
    )

base_preprocessor = make_preprocessor(base_numeric, base_categorical)
X_train_base = base_preprocessor.fit_transform(train)
X_test_base = base_preprocessor.transform(test)
base_feature_names = base_preprocessor.get_feature_names_out()
print("Матрица train:", X_train_base.shape, "; отложенная:", X_test_base.shape)
display(pd.DataFrame(X_train_base[:5, :8], columns=base_feature_names[:8]))

```

	Грейд	Код
0	EN	0
1	MI	1
2	SE	2
3	EX	3

	Размер	Код
0	S	0
1	M	1
2	L	2

	remote_ratio_0	remote_ratio_50	remote_ratio_100
Офис	1	0	0
Гибрид	0	1	0
Удалённо	0	0	1

Матрица train: (11195, 104) ; отложенная: (3781, 104)

	numeric_years_experience	numeric_benefits_score	numeric_job_description_length	numeric_missingindicator_benefits_score	numeric_missingindicator_job_description_length	category_experience_level_EN	category_experience_level_EX	category_experience_level_MI
0	-0.959	-1.584	-0.412	-0.030	-0.028	1.000	0.000	0.000
1	-0.779	1.312	0.815	-0.030	-0.028	0.000	0.000	1.000
2	0.121	0.760	-0.278	-0.030	-0.028	0.000	0.000	0.000

	numeric__years_experience	numeric__benefits_score	numeric__job_description_length	numeric__missingindicator_benefits_score	numeric__missingindicator_job_description_length	category__experience_level_EN	category__experience_level_EX	category__experience_level_MI
3	0.121	-1.102	-1.192	-0.030	-0.028	0.000	0.000	0.000
4	-1.139	0.071	-0.381	-0.030	-0.028	1.000	0.000	0.000

При ONE формате работы у каждой строки один активный столбец. Так модель рассматривает офис, гибридный и удалённую работу как отдельные категории.

Список категорий определяется по обучающей части. При `handle_unknown='ignore'` новая категория получает нули во всём своём блоке. При дальнейшем использовании модели стоит следить, как часто это происходит.

Для обычной линейной регрессии с константой можно убрать одну категорию через `drop='first'`, чтобы избежать линейной зависимости столбцов. В регуляризованной модели можно оставить все категории. Матрицы обеих частей имеют одинаковые столбцы, без пропусков и бесконечных значений.

5. Конструирование признаков

По условию задания необходимо:

- Создать не менее трёх новых признаков на основе имеющихся данных.
- Объяснить, какую информацию добавляет каждый признак и чем он может быть полезен.
- Исследовать связь новых признаков с выбранной целевой переменной.
- Описать закономерности и оценить возможную пользу признаков для будущих моделей.

Ниже созданы шесть признаков и разобраны результаты их проверки. Обучение моделей в этот пункт задания не входит. Поэтому возможное улучшение прогноза рассматривается как гипотеза.

Как проверяются идеи из пункта 3

В маркдаун-ячейке 26 предложены признаки и гипотезы для будущих моделей. Посмотрим, что уже удалось проверить.

1. **Стаж, грейд и бинарный признак.** Сильная связь стажа с грейдом затрудняет интерпретацию коэффициентов линейной модели, но не делает её автоматически некорректной. Флаг `is_senior_or_above` можно сравнить с полным кодированием грейда. Он объединяет EN с MI и SE с EX, поэтому часть различий теряется. Связь со стажем тоже может сохраниться. Здесь создан `senior_large`: старший грейд SE/EX в крупной компании. Его связь с зарплатой разобрана в пунктах 5.2 и 5.3. Отдельный флаг старшего грейда в модели пока не проверялся.
2. **Зарплата относительно среднего по грейду.** Это полезно для поиска необычных вакансий, но среднее по учебному файлу не равно среднему по рынку. Для расчёта отношения нужна сама зарплата, которую предстоит предсказывать. Поэтому такой признак создаёт утечку и в матрицу не включён. В пункте 5.3 сравниваются групповые медианы для описания данных.
3. **Балл соцпакета и длина описания.** Слабая парная корреляция не доказывает бесполезность признака: он может работать вместе с другими. Оба признака сохранены. Исключать их стоит после сравнения моделей с ними и без них на одинаковых разбиениях.
4. **Грейд как главный фактор зарплаты.** Более высокая корреляция не доказывает причинного влияния или преимущества для прогноза. Потребуется сравнить модели со стажем, с грейдом и с обоими признаками. Здесь такой эксперимент не проводится.
5. **PCA для стажа и грейда.** В пункте 5.4 одна компонента сохраняет 96,27% дисперсии стандартизированной пары на `train`. Сжать два признака удалось. Улучшит ли это прогноз зарплаты, ещё нужно проверить: PCA не учитывает зарплату при выборе компонент. Пока оставляем его отдельным исследованием.

5.1. Шесть новых признаков

Все признаки рассчитаны без использования зарплаты. Предполагаем, что прогноз нужен для вакансии, поля которой уже известны.

Признак	Расчёт	Возможная польза и ограничения
application_window_days	Разница между датой окончания приёма и датой публикации, в днях	Плановый срок приёма заявок. Фактический срок найма может отличаться.
skill_count	Число уникальных непустых навыков	Число перечисленных требований. Список может быть неполным.
cross_border	Страны компании и сотрудника различаются	Географическое несовпадение. Гражданство и факт переезда неизвестны.
has_cloud_tools	В списке есть AWS, Azure или GCP	Требование облачных технологий. Уровень владения ими неизвестен.
senior_large	Грейд SE или EX при размере компании L	Сочетание старшего грейда и крупной компании.
years_experience_sq	Квадрат требуемого стажа	Позволяет простой модели описать нелинейную связь со стажем.

Если одна из стран неизвестна, `cross_border` остаётся `NaN`. Иначе сравнение с `Unknown` могло бы ошибочно отметить различие стран. Если страна сотрудника ещё неизвестна при публикации, этот признак нельзя использовать для прогноза в тот момент. Срок приёма заявок тоже должен быть известен заранее.

Цель. Рассчитать шесть признаков и проверить их значения по описательной статистике. Затем перемешать зарплаты: новые признаки должны остаться прежними, поскольку их формулы не используют зарплату.

```
new_columns = [
    "application_window_days",
    "skill_count",
    "cross_border",
    "has_cloud_tools",
    "senior_large",
    "years_experience_sq",
]

def create_features(frame):
    result = frame.copy()
    tokens = frame.required_skills.fillna("").map(
        lambda s: {x.strip().casefold() for x in s.split(",") if x.strip()}
    )
    result["application_window_days"] = (
        frame.application_deadline - frame.posting_date
    ).dt.days
    result["skill_count"] = tokens.map(len)
    countries_known = frame.company_location.notna() & frame.employee_residence.notna()
    result["cross_border"] = np.where(
        countries_known,
        (frame.company_location != frame.employee_residence).astype(float),
        np.nan,
    )
    result["has_cloud_tools"] = tokens.map(lambda s: int(bool(s & {"aws", "azure", "gcp"})))
    result["senior_large"] = (
        frame.experience_level.isin(["SE", "EX"]) & frame.company_size.eq("L")
    ).astype(int)
    result["years_experience_sq"] = frame.years_experience**2
    return result

features = create_features(checkered)
feature_eda = features.loc[features.salary_usd.notna()]
train_fe = create_features(train)
test_fe = create_features(test)
display(features[new_columns].describe().T)
display(features[new_columns].isna().rename_axis("index").sum().to_frame("Пропуски"))
skill_counts = features.skill_count.value_counts().sort_index()
display(skill_counts.rename("Вакансий"))
# Проверка отсутствия зависимости производных признаков от целевой переменной.
```

```

shuffled_target = checked.copy()
shuffled_target["salary_usd"] = np.random.default_rng(RANDOM_STATE).permutation(
    checked.salary_usd
)
features_after_shuffle = create_features(shuffled_target)[new_columns]
features_ignore_salary = features_after_shuffle.equals(features[new_columns])
print("Новые признаки не зависят от перемешивания зарплаты:", features_ignore_salary)

```

	count	mean	std	min	25%	50%	75%	max
application_window_days	15,000.000	43.546	17.325	14.000	29.000	44.000	59.000	74.000
skill_count	15,000.000	3.993	0.816	3.000	3.000	4.000	5.000	5.000
cross_border	14,997.000	0.284	0.451	0.000	0.000	0.000	1.000	1.000
has_cloud_tools	15,000.000	0.388	0.487	0.000	0.000	0.000	1.000	1.000
senior_large	15,000.000	0.167	0.373	0.000	0.000	0.000	0.000	1.000
years_experience_sq	15,000.000	69.856	97.609	0.000	4.000	25.000	100.000	361.000

	Пропуски
application_window_days	0
skill_count	0
cross_border	3
has_cloud_tools	0
senior_large	0
years_experience_sq	0

```

skill_count
3    5050
4    5007
5    4943
Name: Вакансий, dtype: int64

```

Новые признаки не зависят от перемешивания зарплаты: True

Приём заявок длится от 14 до 74 дней. В `required_skills` указано от трёх до пяти навыков. Значит, «Топ 5» в словаре не означает, что в каждой строке ровно пять записей. При подсчёте удаляем повторы внутри списка. Также учитываем различия в регистре и пробелах.

Квадрат стажа сохраняет порядок наблюдений. Его ранговая связь с зарплатой не показывает, добавляет ли он пользу по сравнению с исходным стажем. После перемешивания зарплаты все шесть признаков остались прежними. Это подтверждает, что расчёт не использует целевую переменную.

5.2. Связь новых признаков с зарплатой

Для числовых признаков с повторяющимися значениями используем коэффициент Кендалла τ -b. Для бинарных сравниваем распределения зарплаты тестом Манна–Уитни и рассчитываем рангово-бисериальную корреляцию. Положительный знак означает, что при значении признака 1 зарплата чаще выше. Тест Манна–Уитни без дополнительных условий нельзя считать проверкой равенства медиан.

Уровень значимости: 0,05. Проверки двусторонние, с поправкой Холма на шесть сравнений. Берём пары с известными значениями. Коэффициенты разных типов напрямую по величине не сравниваем.

Вакансии одной компании могут быть связаны, хотя расчёт p -value предполагает независимость строк. Поэтому осторожно переносим выводы за пределы файла. Отсутствие значимости не доказывает отсутствия связи.

Цель. Оценить связь новых признаков с зарплатой выбранными методами. Таблица показывает размеры выборок, эффекты и p-value. Поправка Холма учитывает шесть одновременных проверок.

```
binary = ["cross_border", "has_cloud_tools", "senior_large"]
fe_rows = []
for col in new_columns:
    pair = feature_eda[[col, "salary_usd"]].dropna()
    if col in binary:
        one = pair.loc[pair[col] == 1, "salary_usd"]
        zero = pair.loc[pair[col] == 0, "salary_usd"]
        u, p = stats.mannwhitneyu(one, zero, alternative="two-sided", method="asymptotic")
        effect = 2 * u / (len(one) * len(zero)) - 1
        method = "Манн-Уитни / r_rb"
        delta = one.median() - zero.median()
    else:
        effect, p = stats.kendalltau(pair[col], pair.salary_usd)
        method = "Кендалл τ-b"
        delta = np.nan
    fe_rows.append(
        {
            "Признак": col,
            "n": len(pair),
            "Метод / эффект": method,
            "Эффект": effect,
            "p": p,
            "Δ медиан 1-0, USD": delta,
        }
    )
fe_associations = pd.DataFrame(fe_rows).set_index("Признак")
fe_associations["p Holm"] = holm(fe_associations.p)
show_stats(fe_associations, ["p", "p Holm"])
```

	n	Метод / эффект	Эффект	p	Δ медиан 1-0, USD	p Holm
Признак						
application_window_days	14 976	Кендалл τ-b	0.004	0.46	nan	1
skill_count	14 976	Кендалл τ-b	-0.010	0.105	nan	0.42
cross_border	14 973	Манн-Уитни / r_rb	0.009	0.393	608.000	1
has_cloud_tools	14 976	Манн-Уитни / r_rb	-0.007	0.449	-1 300.000	1
senior_large	14 976	Манн-Уитни / r_rb	0.666	<1e-300	73 436.000	<1e-300
years_experience_sq	14 976	Кендалл τ-b	0.599	<1e-300	nan	<1e-300

- application_window_days: эффект 0.004, p Holm 1; значимой связи не обнаружено.
- skill_count: эффект -0.010, p Holm 0.42; значимой связи не обнаружено.
- cross_border: эффект 0.009, p Holm 1; значимой связи не обнаружено.
- has_cloud_tools: эффект -0.007, p Holm 1; значимой связи не обнаружено.
- senior_large: эффект 0.666, p Holm <1e-300; связь статистически значима.
- years_experience_sq: эффект 0.599, p Holm <1e-300; связь статистически значима.

Для срока, числа навыков, различия стран и облачных технологий оценки связи близки к нулю. Значимой связи в этом файле не обнаружено. В другой выборке результат может отличаться.

senior_large объединяет грейд и размер компании. Оба исходных признака уже связаны с зарплатой. Высокая оценка связи ожидаема. Добавляет ли новый признак пользу к исходным, ещё нужно проверить. Квадрат стажа позволяет менять форму зависимости, но не добавляет новых исходных данных.

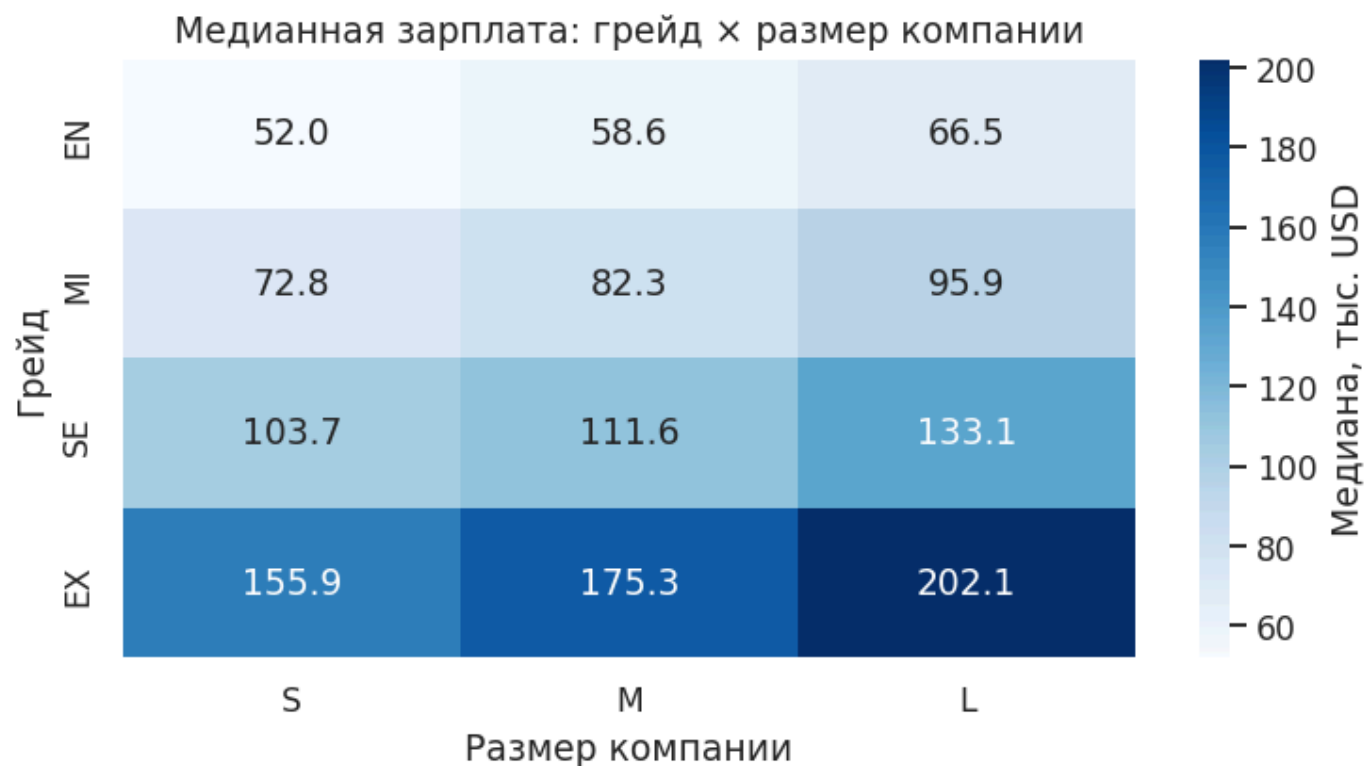
5.3. Зарплата по грейду и размеру компании

Для проверки `senior_large` сравним медианные зарплаты во всех 12 сочетаниях грейда и размера компании. Тепловая карта покажет, сохраняется ли разница между размерами компаний внутри каждого грейда.

Цель. Разобраться в связи `senior_large` с зарплатой. Для сочетаний грейда и размера компании рассчитываются число вакансий и медианная зарплата. Тепловая карта помогает сравнить группы.

```
grade_size = feature_eda.pivot_table(
    index="experience_level", columns="company_size", values="salary_usd", aggfunc="median"
).reindex(index=LEVELS, columns=SIZES)
grade_size_n = pd.crosstab(feature_eda.experience_level, feature_eda.company_size).reindex(
    index=LEVELS, columns=SIZES
)
display(grade_size_n)
fig, ax = plt.subplots(figsize=(7, 4))
sns.heatmap(
    grade_size / 1000,
    annot=True,
    fmt=".1f",
    cmap="Blues",
    ax=ax,
    cbar_kws={"label": "Медиана, тыс. USD"},
)
ax.set(
    title="Медианная зарплата: грейд × размер компании",
    xlabel="Размер компании",
    ylabel="Грейд",
)
finish()
large_premium = grade_size["L"] / grade_size["S"] - 1
display((large_premium * 100).rename("Разница медиан L к S, %").to_frame())
```

company_size	S	M	L
experience_level			
EN	1265	1236	1212
MI	1218	1276	1282
SE	1224	1216	1295
EX	1294	1259	1199



	Разница медиан L к S, %
experience_level	
EN	27.838
MI	31.818
SE	28.355
EX	29.678

Медианная зарплата в крупных компаниях выше, чем в малых, на 27,8–31,8% внутри каждого грейда. Это описательная разница. Отдельный совместный эффект грейда и размера компании здесь не проверялся.

Чтобы оценить пользу `senior_large`, нужно сравнить модели с этим признаком и без него на одинаковых разбиениях. Пока считать его улучшением рано.

5.4. Дополнительно: проверка PCA по лекции 4

Применим PCA к стажу, грейду, размеру компании, баллу соцпакета и длине описания. Грейд и размер кодируем порядковыми числами. Это условно задаёт расстояния между категориями, что важно помнить при интерпретации. Зарплата в PCA не участвует.

Заполнение медианой, стандартизация и PCA обучаются на `train`. Таблица и график каменистой осыпи помогут выбрать число компонент: по правилу Кайзера и порогу 95% сохранённой дисперсии. Нагрузки в отдельной таблице показывают корреляции признаков с компонентами. Дополнительно проверим, сколько информации о разбросе стажа и грейда сохраняет одна компонента.

Цель. Проверить идею сжатия коррелирующих признаков с помощью PCA. После заполнения пропусков и стандартизации на `train` рассчитываются доли сохранённой дисперсии и нагрузки. Отдельно исследуется одна компонента для пары «стаж и грейд».

```
def pca_frame(frame):
    return frame.assign(
        exp_num=frame.experience_level.map(LEVEL_MAP),
        size_num=frame.company_size.map(SIZE_MAP),
```

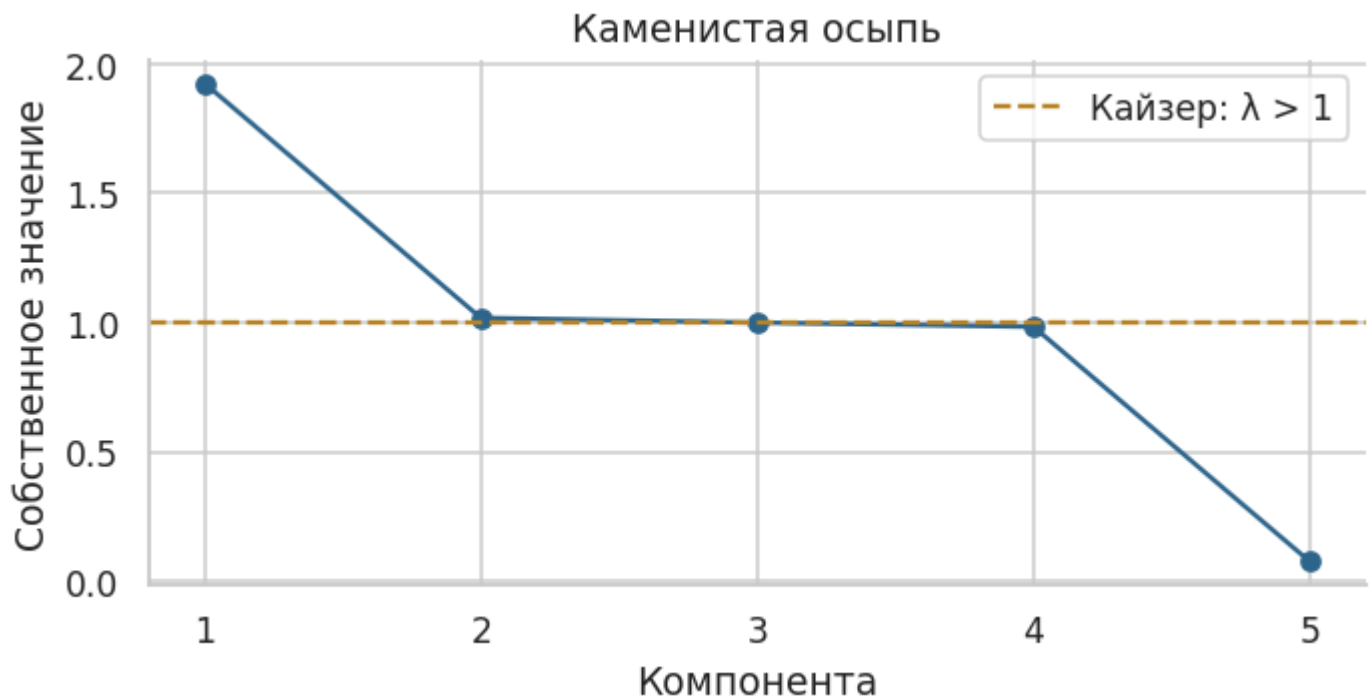
```

)

pca_columns = [
    "years_experience",
    "exp_num",
    "size_num",
    "benefits_score",
    "job_description_length",
]
pca_pipeline = Pipeline(
    [
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("pca", PCA(svd_solver="full")),
    ]
)
pca_train = pca_pipeline.fit_transform(pca_frame(train)[pca_columns])
pca_test = pca_pipeline.transform(pca_frame(test)[pca_columns])
pca = pca_pipeline.named_steps["pca"]
eigenvalues = pca.explained_variance_
variance = pca.explained_variance_ratio_
cumulative = np.cumsum(variance)
component_names = [f"PC{i + 1}" for i in range(len(pca_columns))]
n95 = int(np.searchsorted(cumulative, 0.95) + 1)
n_kaiser = int((eigenvalues > 1).sum())
display(
    pd.DataFrame(
        {
            "Собственное значение": eigenvalues,
            "Доля дисперсии": variance,
            "Накопленная доля": cumulative,
        },
        index=component_names,
    )
)
fig, ax = plt.subplots(figsize=(6.5, 3.5))
ax.plot(range(1, 6), eigenvalues, "o-", color=BLUE)
ax.axhline(1, color=GOLD, linestyle="--", label="Кайзер:  $\lambda > 1$ ")
ax.set(
    xticks=range(1, 6),
    xlabel="Компонента",
    ylabel="Собственное значение",
    title="Каменистая осыпь",
)
ax.legend()
finish()
z_train = pca_pipeline.named_steps["scaler"].transform(
    pca_pipeline.named_steps["imputer"].transform(pca_frame(train)[pca_columns])
)
loadings = np.corrcoef(z_train.T, pca_train.T)[: len(pca_columns), len(pca_columns) :]
loading_table = pd.DataFrame(loadings, index=pca_columns, columns=component_names)
display(loading_table)
pair_pca = Pipeline(
    [("scaler", StandardScaler()), ("pca", PCA(n_components=1, svd_solver="full"))]
)
pair_train = pair_pca.fit_transform(pca_frame(train)[["years_experience", "exp_num"]])
pair_test = pair_pca.transform(pca_frame(test)[["years_experience", "exp_num"]])
pair_retained = pair_pca.named_steps["pca"].explained_variance_ratio_[0]
print("Доля дисперсии пары стаж и грейд в одной компоненте, %:", round(pair_retained * 100, 2))

```

	Собственное значение	Доля дисперсии	Накопленная доля
PC1	1.926	0.385	0.385
PC2	1.018	0.204	0.589
PC3	0.999	0.200	0.789
PC4	0.983	0.197	0.985
PC5	0.074	0.015	1.000



	PC1	PC2	PC3	PC4	PC5
years_experience	0.981	0.002	-0.005	0.006	0.193
exp_num	0.981	-0.002	-0.005	0.005	-0.193
size_num	-0.003	-0.636	0.447	0.629	0.000
benefits_score	-0.012	-0.337	-0.894	0.295	0.000
job_description_length	0.008	-0.707	0.025	-0.707	0.001

Доля дисперсии пары стаж и грейд в одной компоненте, %: 96.27

Две компоненты сохраняют 58,86% дисперсии пяти признаков. Для порога 95% нужны четыре компоненты, правило Кайзера оставляет две. Одна компонента пары «стаж и грейд» сохраняет 96,27% её дисперсии.

Первая компонента в основном отражает опыт: стаж и грейд имеют большие нагрузки одного знака. Другие компоненты связаны с размером компании, соцпакетом и длиной описания. Некоторые собственные значения близки к 1, поэтому результат правила Кайзера может меняться от выборки к выборке.

Сжатие всех пяти признаков до двух теряет заметную часть разброса. Для будущей модели интереснее проверить одну компоненту стажа и грейда. В основную матрицу её пока не включаем.

Как сказано на слайде 30 лекции 4, «высокая вариативность $\neq$ высокая смысловая важность для конкретной исследовательской задачи». Поэтому сохранённая дисперсия не доказывает пользу для прогноза зарплаты. Красивое сжатие получилось, но место в итоговой модели ему ещё нужно заслужить.

5.5. Итоговые матрицы и проверки

Добавляем шесть новых признаков к исходным. Обучаем `ColumnTransformer` на обучающей части. Затем применяем его к отложенным данным и строкам без известной зарплаты. PCA в этот вариант не входит. Его можно будет сравнить с основной обработкой в отдельном эксперименте.

Цель. Собрать итоговые матрицы. Обработчик обучается на `train` и применяется к остальным строкам. Таблица проверок показывает сохранность CSV, учёт строк, разделение компаний и корректность матриц.

```
final_preprocessor = make_preprocessor(base_numeric + new_columns, base_categorical)
X_train = final_preprocessor.fit_transform(train_fe)
```

```

X_test = final_preprocessor.transform(test_fe)
X_unknown_target = final_preprocessor.transform(create_features(unlabeled))
y_train = train.salary_usd.copy()
y_test = test.salary_usd.copy()

# Короткие проверки с понятными названиями.
matrices = [X_train, X_test, X_unknown_target]
selected_columns = base_numeric + new_columns + base_categorical
companies_overlap = train.company_name.isin(test.company_name).any()
jobs_overlap = train.job_id.isin(test.job_id).any()
train_target_unchanged = y_train.equals(raw.loc[train.index, "salary_usd"])
test_target_unchanged = y_test.equals(raw.loc[test.index, "salary_usd"])
csv_unchanged = source_sha256 == hashlib.sha256(DATA_PATH.read_bytes()).hexdigest()

checks = dict(data_checks)
checks.update(
    {
        "Рабочая копия создана из исходного CSV": source_matches,
        "Исходный CSV не изменён": csv_unchanged,
        "Все строки учтены": len(train) + len(test) + len(unlabeled) == len(raw),
        "Нет пересечений компаний": not companies_overlap,
        "Нет пересечений job_id": not jobs_overlap,
        "Матрицы без NaN и inf": all(np.isfinite(matrix).all() for matrix in matrices),
        "Одинаковое число столбцов": X_train.shape[1]
        == X_test.shape[1]
        == X_unknown_target.shape[1],
        "Зарплата и идентификаторы исключены": not any(
            column in selected_columns for column in ["salary_usd", "job_id", "company_name"]
        ),
        "Новые признаки не используют зарплату": features_ignore_salary,
        "Наблюдаемые зарплаты не заменены": train_target_unchanged and test_target_unchanged,
    }
)
check_results = pd.Series(checks, name="Проверка пройдена")
display(check_results.to_frame())
if not check_results.all():
    raise ValueError("Некоторые проверки не пройдены. См. таблицу выше.")

display(
    pd.DataFrame(
        {
            "Часть": ["Обучающая", "Отложенная", "Без известной зарплаты"],
            "Строки": [len(X_train), len(X_test), len(X_unknown_target)],
            "Признаки": [X_train.shape[1], X_test.shape[1], X_unknown_target.shape[1]],
        }
    )
)

```

	Проверка пройдена
Нет повторов job_id	True
Грейды есть в словаре	True
Размеры компаний есть в словаре	True
Формат работы имеет известный код	True
Уровни образования есть в словаре	True
Стаж находится между 0 и 60 годами	True
Дедлайн не раньше публикации	True
Рабочая копия создана из исходного CSV	True
Исходный CSV не изменён	True
Все строки учтены	True
Нет пересечений компаний	True
Нет пересечений job_id	True
Матрицы без NaN и inf	True
Одинаковое число столбцов	True

	Проверка пройдена
Зарплата и идентификаторы исключены	True
Новые признаки не используют зарплату	True
Наблюдаемые зарплаты не заменены	True

	Часть	Строки	Признаки
0	Обучающая	11195	111
1	Отложенная	3781	111
2	Без известной зарплаты	24	111

Подготовлены матрицы (11195, 111) (обучающая часть), (3781, 111) (отложенная часть) и (24, 111) (неизвестная цель); все проверки выполнены.

Выводы по пунктам 4–5

Даты и категории приведены к единому виду, подозрительные значения обработаны по описанным правилам. Заполнение пропусков и масштабирование обучены только на train. Для двух числовых признаков сравнены MinMaxScaler, StandardScaler, RobustScaler, log1p и sqrt. Для категорий показаны порядковое кодирование и ONE. Строки без используемой зарплаты сохранены отдельно. Целевые значения не придумывались.

Созданы шесть признаков и проверена их связь с зарплатой с поправкой Холма. Расчёт не использует целевую переменную. Пользу для прогноза ещё предстоит проверить. senior_large объединяет уже связанные с зарплатой грейд и размер компании. Квадрат стажа меняет форму зависимости, сохраняя порядок наблюдений.

Стаж и грейд удалось сжать в одну компоненту. Для всех пяти признаков сжатие до двух компонент теряет заметную долю дисперсии. Поэтому PCA пока оставлен отдельным исследованием.

Проверена чувствительность к границам зарплаты, изменения записаны в журнал. Выводы относятся к этому учебному файлу. Причинные связи и улучшение прогноза здесь не проверялись.

Источники методов

Для пунктов 4–5 использованы PDF лекций 1–4 из папки курса: лекция1_Intro.pdf (93 слайда), лекция2_EDA.pdf (67), лекция3_FE.pdf (49), лекция4_DR.pdf (64). Номера основных слайдов приведены в начале пункта 4. Также использованы примеры масштабирования, корреляций и PCA из семинаров 2–4. Все видеозаписи лекций просмотрены нами. Материалы лекций также самостоятельно изучены на парах и после них.

Справочные материалы: SciPy: Kendall τ , scikit-learn: StandardScaler, scikit-learn: PCA, поправка Холма.

Использование генеративных моделей

Мы любим ИИ и часто им пользуемся, но к его ответам относимся осторожно. Для проверки результатов и редактирования использовали OpenAI Codex (GPT-6 Astra), Claude Opus 5 и Qimi K3. Мы лично разобрали и проверили весь код. При его подготовке использовали ИИ, а при выборе методов руководствовались лекциями. ИИ помог нам разобраться в библиотеке pandas и в работе с данными. Не скрываем, что большую часть кода разбирали и проверяли с помощью Codex и других моделей. Это помогало понять, как работает код, и находить простые ошибки. В Codex запускали расчёты и проверяли, соответствуют ли выводы таблицам и графикам. ИИ также помогал с подготовкой кода и черновых пояснений, в том числе в пунктах 4 и 5. В тексте есть наши формулировки и варианты, предложенные ИИ, которые затем редактировались. Исходные данные не генерировали.

Perplexity использовали для поиска справочной информации. Материалы лекций загрузили в NotebookLM для более простого усвоения, все материалы самостоятельно просмотрели как на парах, так и после них. Ответы моделей сопоставляли с лекциями и результатами расчётов.

Примеры формулировок запросов:

- «Покажи, как заполнить пропуски медианами обучающей выборки и применить те же значения к новым данным».

- «Сопоставь выводы с таблицами и графиками. Найди утверждения, которые не подтверждаются результатами».
- «Прочитай весь текст. Убери повторы и шаблонные обороты, сократи длинные предложения. Сохрани смысл и числа».
- «Проверь орфографию, пунктуацию и согласование слов. Не меняй условия задания».

Это примеры формулировок, а не дословный журнал переписки.

Мы надеемся, что проделанная работа вам понравится и вы оцените пасхалочки, которые мы оставили в этой работе 😊